

Linux Terminal Cheat Sheet — Basics

A quick-reference for everyday terminal commands. Keep this open while you work — you don't need to memorize any of this, just know it's here to check.

Where am I / what's here

Command	What it does
<code>pwd</code>	Print current directory (where you are right now)
<code>ls</code>	List files/folders in the current directory
<code>ls -l</code>	List with details (permissions, size, date)
<code>ls -a</code>	List including hidden files (start with <code>.</code>)
<code>ls -lh</code>	Details, with human-readable sizes (KB/MB/GB instead of bytes)

Moving around

Command	What it does
<code>cd foldername</code>	Move into a folder
<code>cd ..</code>	Move up one level (to the parent folder)
<code>cd ~</code> or just <code>cd</code>	Jump straight to your home folder
<code>cd -</code>	Jump back to the previous folder you were in
<code>cd /full/path/here</code>	Jump directly to an exact location

Tip: type the first few letters of a folder/file name, then press **Tab** — the terminal will auto-complete it (and automatically handle spaces in names for you). Press Tab twice if there are multiple matches, to see a list.

Creating things

Command	What it does
<code>mkdir foldername</code>	Create a new folder
<code>mkdir -p path/to/nested/folder</code>	Create nested folders in one go, even if the parent folders don't exist yet
<code>touch filename.txt</code>	Create a new empty file

Copying, moving, renaming

Command	What it does
<code>cp source destination</code>	Copy a file
<code>cp -r source destination</code>	Copy a folder and everything inside it (the <code>-r</code> means "recursive")
<code>mv source destination</code>	Move a file or folder
<code>mv oldname newname</code>	Rename a file or folder (yes, renaming is just "moving" to a new name in the same place)

Examples:

```
cp report.txt ~/Documents/
cp -r MyProject/ ~/Backups/
mv draft.txt final.txt
```

Deleting things

Command	What it does
<code>rm filename</code>	Delete a file
<code>rm -r foldername</code>	Delete a folder and everything inside it
<code>rmdir foldername</code>	Delete a folder, but only if it's empty

⚠ **Important:** there is no Recycle Bin / Trash in the terminal. `rm` deletes immediately and permanently. Double-check the filename before pressing Enter, especially with `-r`.

Handling spaces in names

Linux terminals treat a space as "end of this argument, start of the next one" — so a name like `My Documents` needs special handling:

```
cd "My Documents"           # wrap it in quotes
cd My\ Documents            # or escape the space with a backslash
```

Using **Tab-completion** avoids this problem entirely — it adds the escaping for you automatically.

Viewing file contents

Command	What it does
<code>cat filename</code>	Print the whole file to the screen at once
<code>less filename</code>	View a file page-by-page (press <code>q</code> to quit, arrow keys to scroll)
<code>head filename</code>	Show just the first 10 lines
<code>tail filename</code>	Show just the last 10 lines
<code>nano filename</code>	Open the file in a simple text editor

Finding things

Command	What it does
<code>find / -iname "filename"</code>	Search the whole system for a file by name (case-insensitive)
<code>grep "text" filename</code>	Search inside a file for lines containing that text
<code>grep -n "text" filename</code>	Same, but also show the line number of each match
<code>grep -r "text" foldername</code>	Search inside every file in a folder

Checking what's running

Command	What it does
<code>top</code>	Live view of running processes and resource usage (press <code>q</code> to quit)
<code>htop</code>	A nicer-looking version of <code>top</code> (may need <code>sudo apt install htop</code> first)
<code>ps aux</code>	List all currently running processes

Command	What it does
<code>ps aux grep name</code>	List running processes, filtered to ones matching "name"

Stopping a running program

Action	How
Stop the program currently running in this terminal	<code>Ctrl + C</code>
Pause it (can resume later with <code>fg</code>)	<code>Ctrl + Z</code>
Force-kill a stuck process by name	<code>pkill processname</code>
Force-kill by process ID (PID, from <code>ps</code> or <code>top</code>)	<code>kill PID</code> or <code>kill -9 PID</code> (the <code>-9</code> is a harder force-kill)

Permissions & running as admin

Command	What it does
<code>sudo command</code>	Run a single command with admin/root privileges (you'll be asked for your password)
<code>chmod +x filename</code>	Make a file executable (runnable as a program/script)

Installing software (Debian/Kali-based systems)

Command	What it does
<code>sudo apt update</code>	Refresh the list of available packages (do this before installing anything)
<code>sudo apt install packagename</code>	Install a package
<code>sudo apt remove packagename</code>	Uninstall a package

Compressed files / archives

Command	What it does
<code>tar -xzvf archive.tar.gz</code>	Extract a <code>.tar.gz</code> archive
<code>unzip archive.zip</code>	Extract a <code>.zip</code> archive

Useful habits

- **Tab-completion**: type a few letters, hit `Tab` . Saves typing and avoids typos in long paths.
- **Up arrow**: cycles back through your previously typed commands — no need to retype something you just ran.
- **Ctrl + R** : search backward through your command history — start typing part of a past command and it'll find it.
- **clear** : clears the terminal screen (doesn't affect anything running, just tidies the view).
- **Two dots** `..` : means "the parent folder," useful in paths like `cp ../file.txt .` (copy a file from one level up into the current folder — note `.` alone means "here").

Quick path reminders

Symbol	Meaning
<code>~</code>	Your home folder (e.g. <code>/home/toonz</code>)
<code>.</code>	The current folder
<code>..</code>	The parent (one level up) folder
<code>/</code>	The root of the whole filesystem